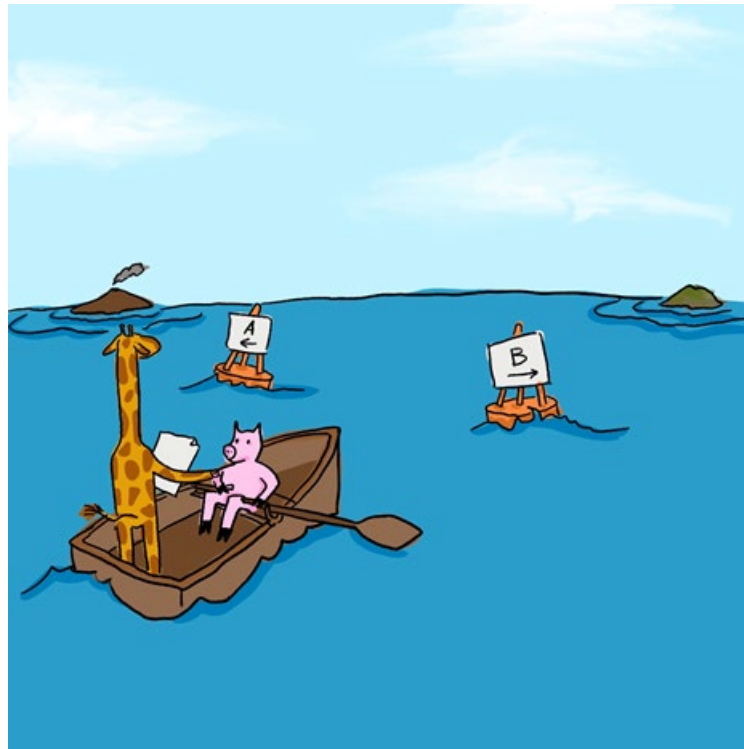


# Computability: Formal Languages and Deterministic Finite Automata



(art credit: CMU CS251)

“The question of whether a computer can think is no more interesting than the question of whether a submarine can swim.”  
- Edsger Dijkstra

# Today is International Right to Know Day

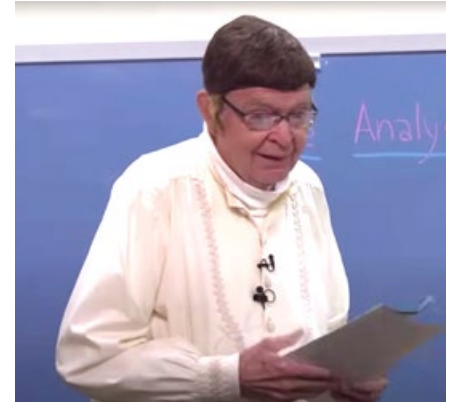
- Other Holidays:
  - Family Day
  - Read a Child a Book You Like Day
  - British Home Child Day
  - World Rabies Day
  - Gold Lining Day
  - National Good Neighbor Day
  - National Strawberry Cream Pie Day
  - Fish Tank Floorshow Night
- This week is National Women's Health and Fitness Week
- September is National Save a Tiger Month

# So far: Analysis of Algorithms



Don Knuth

“Father of Analysis of Algorithms”



---

Next up: **Computability**

- What is a computer?
- What problems can a computer solve?
- What problems can a computer **not** solve?

# Techniques and Paradigms in this Course

- Divide-and-conquer, greed, dynamic programming, the power of randomness  
 Problems that are **easy** for a computer
- Computability  Problems that are **impossible** for a computer
- NP-completeness and approximation algorithms
- Cryptography  
 Problems that are “**probably hard**” for a computer  
 Using “probably hard” problems for our benefit (hiding secrets)

**Wait... what *is* a computer, anyway?**

An algorithm “solves” a problem  
if it outputs a correct solution  
on every input instance.

Each of these  
underlined words  
has a precise  
definition!



We'll define last 3 terms now.  
We'll save algorithm for later.

What is a computational **problem**?

We'll start with some examples.

Example problem:

# MULTIPLICATION

## ***Instance***

(also known as ***input***)

3, 7

610, 25

50, 610

15251, 252

12345679, 9

## ***Solution***

(a.k.a. ***output***)

21

15250

30500

3843252

111111111

A **problem** is a collection of  
**instances** and a **solution** to  
each instance.

Example problem:

# PALINDROME?

## ***Instance***

(also known as ***input***)

a

10101

selfless

huh

376

race car

ten animals i slam in a net

## ***Solution***

(a.k.a. ***output***)

Yes

Yes

No

Yes

No

Yes

Yes (!)

**PALINDROME?** is a decision problem:  
a problem where the solution is **Yes / No**.

(Also known as **True / False**,  
**1 / 0**,  
**accept / reject**)

# Representing problems

The instances of a problem can be (depending on the problem):

- numbers: 0, 1, 281, 376, ...
- strings: “hello”, “world”, “376”, ...
- pairs/tuples of numbers: (0,1), (3,7,6), ...
- lists of strings
- graphs
- images
- ...

All these can be *encoded* by **strings**.

Even just by **binary** (0/1) strings!

Binary is easy  
when you have  
only 2 “toes” on  
each hoof!



# Alphabets, Strings, and Languages

**Alphabet:** A nonempty *finite* set  $\Sigma$  of symbols (characters).

$\Sigma = \{0,1\}$  is the default choice.

**String:** A *finite* sequence of zero or more symbols.

(or “word”)

The *empty* string (having zero symbols) is denoted  $\epsilon$ .

For any  $a \in \Sigma$ :

$a^k$  means  $k$   $a$ 's

$a^*$  means  $\geq 0$   $a$ 's

$a^+$  means  $\geq 1$   $a$ 's

$\Sigma^k$  = set of length- $k$  strings over  $\Sigma$ .

$\Sigma^*$  = set of **all** (finite-length) strings over  $\Sigma$ .

$\Sigma^+$  = set of all nonempty strings over  $\Sigma$ .

For any  $x, y$  :  $x|y$  means  $x$  OR  $y$

**Language:** A collection (set) of strings.

I.e., any subset  $L \subseteq \Sigma^*$ .

The empty language (set) is denoted  $\emptyset$ .

# Examples of Languages

$L = a^* = \{\epsilon, a, aa, aaa, \dots\}$   $\{0, 01, 011, 0111, \dots\}$

$L = 01^* =$  (set of) all strings with one 0, followed by zero or more 1's

$L = (01)^* =$  (set of) all strings with 0 or more "01"s:  $\{\epsilon, 01, 0101, \dots\}$

$L = \{x^k y^k : k \geq 0\} =$  (set of) all strings consisting of some number of x's followed by the same number of y's  $\{\epsilon, xy, xxxy, \dots\}$

**Question:** Suppose  $L = \{\epsilon\}$ . Does  $L = \emptyset$ ?

**Answer:** No:  $L$  has 1 element (the empty string);  $\emptyset = \{\}$  has 0 elts.

**Question:** Does  $a^*$  have infinite strings? infinitely many strings?

**Answer:** No, and Yes: every individual string has *finite length*;  
 $a^*$  has *infinitely many* such strings.

**Question:** (How) is  $a^*|b^*$  different from  $(a|b)^*$  ?

**Answer:**  $a^*|b^*$  only has strings with all a's or all b's  
 $(a|b)^*$  has strings like abbaabaa

# Representing Problems

We can *encode* instances and solutions as *strings*.

Example problem:

# MULTIPLICATION

## ***Instance***

(also known as ***input***)

3, 7

610, 25

50, 610

15251, 252

12345679, 9

## ***Solution***

(a.k.a. ***output***)

21

15250

30500

3843252

111111111

# Representing Problems

We can *encode* instances and solutions as *strings*.

Thus, we can view a **problem** as a **function** (or relation)

$$f : \Sigma^* \rightarrow \Sigma^*$$

mapping each instance to its solution.

So, a **decision problem** can be seen as a function

$$f : \Sigma^* \rightarrow \{\text{No}, \text{Yes}\}$$

# Representing problems

A **decision problem** can be seen as a function

$$f : \Sigma^* \rightarrow \{\text{No}, \text{Yes}\}$$

or equivalently, as a **language**

$$L \subseteq \Sigma^*$$

$$L = \{x \in \Sigma^* : f(x) = \text{Yes}\} \quad f(x) = \begin{cases} \text{Yes} & \text{if } x \in L \\ \text{No} & \text{if } x \notin L \end{cases}$$

E.g.:  $L_{\text{PALINDROME}} = \{x \in \Sigma^* : x \text{ is a palindrome}\}$

What is **computation**?

What is an **algorithm**?

This lecture:

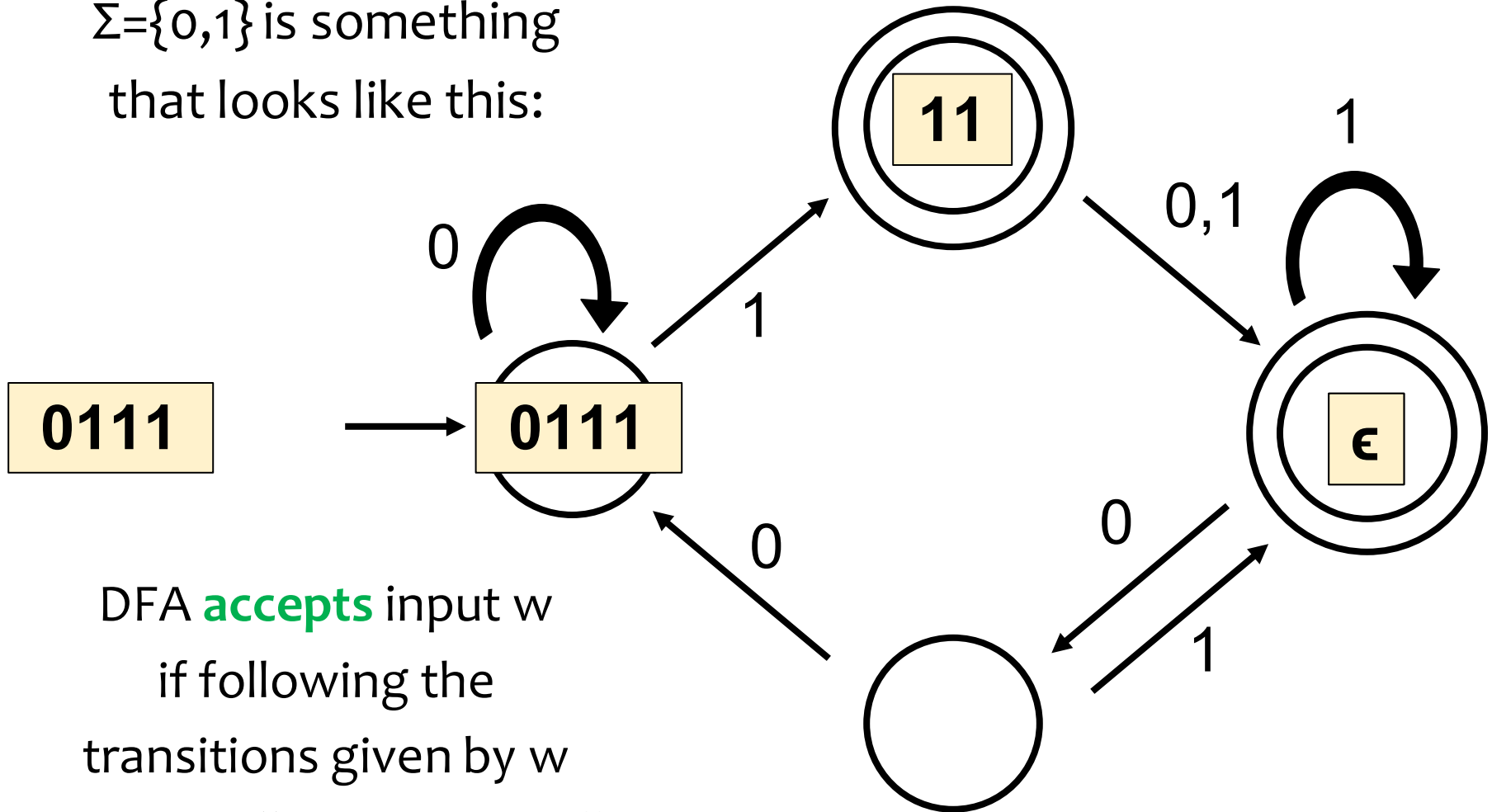
A very simple (but still useful!) computational  
model:

**Deterministic Finite Automata (DFA)**

A good warmup before we study general  
models of computations next lecture.

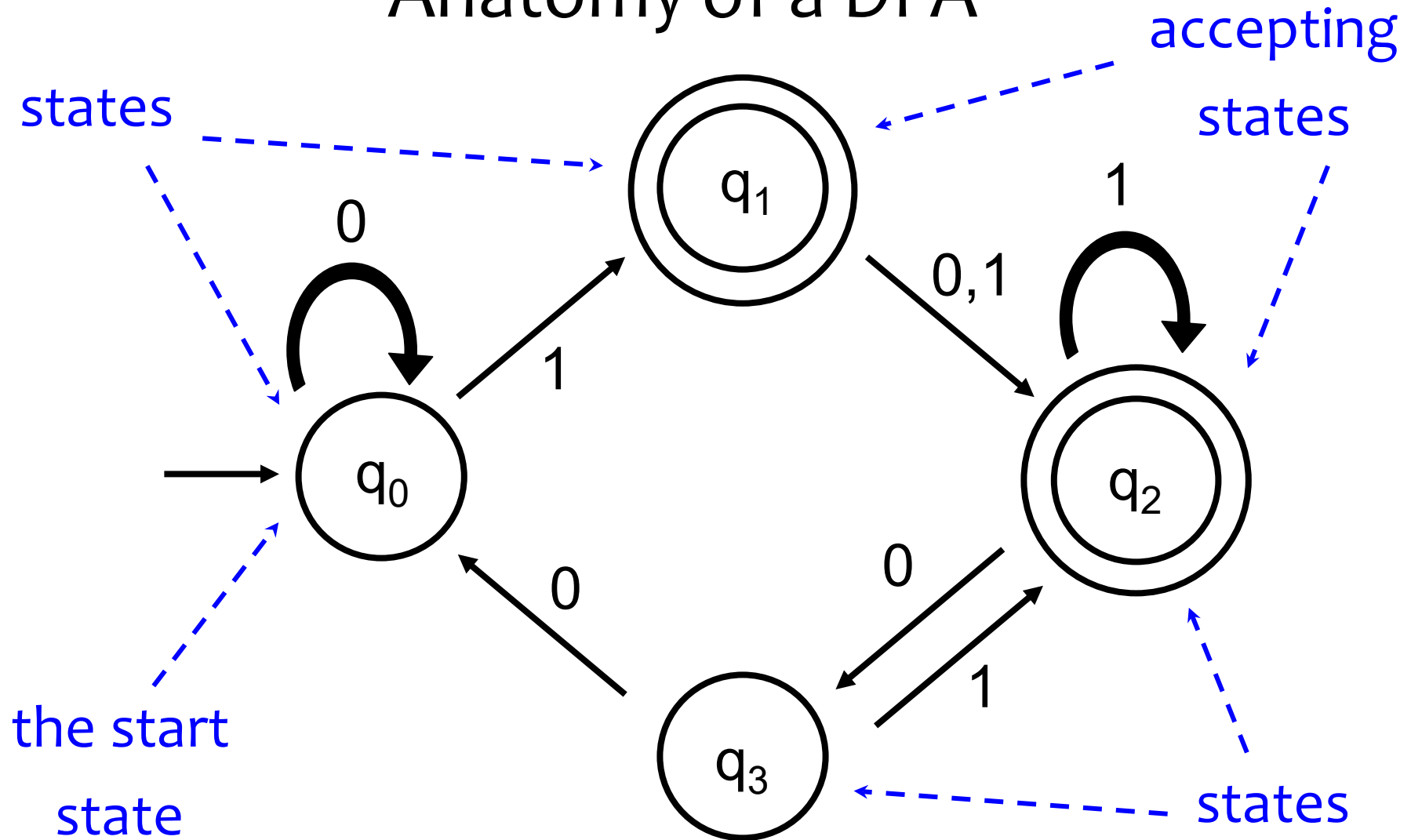
# Deterministic Finite Automata (DFAs)

A DFA over alphabet  $\Sigma=\{0,1\}$  is something that looks like this:



DFA **accepts** input  $w$   
if following the  
transitions given by  $w$   
ends in a “double circle.”

# Anatomy of a DFA



transition rules: the labeled arrows  
(one arrow per symbol, per state)

# Computing with DFAs

Let  $M$  be a DFA with alphabet  $\Sigma$ .

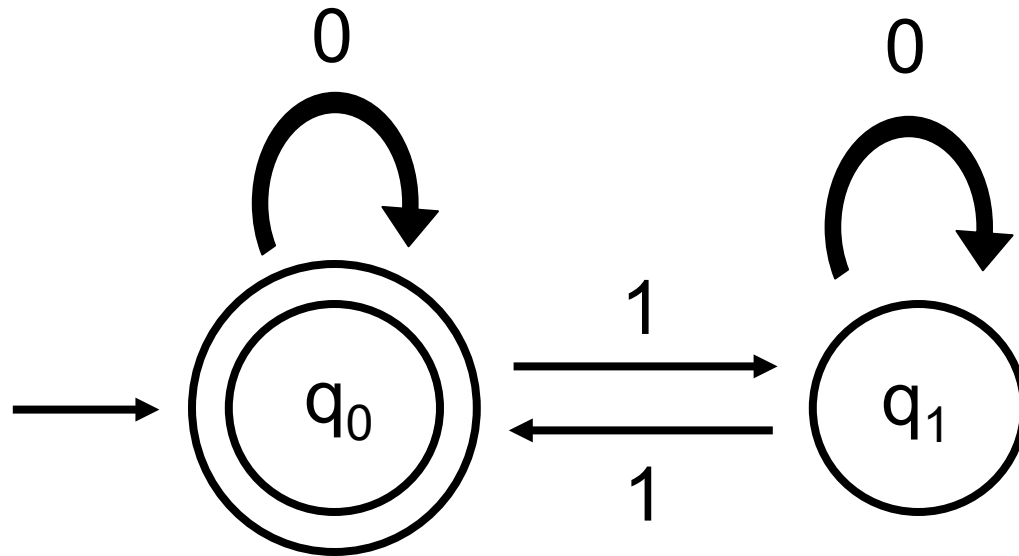
$M$  **accepts** some strings in  $\Sigma^*$ , and **rejects** the rest.

**Definition:**  $L(M) = \{x \in \Sigma^* : M \text{ accepts } x\}$

Called the “language decided by  $M$ ”.

If  $L$  is a language,

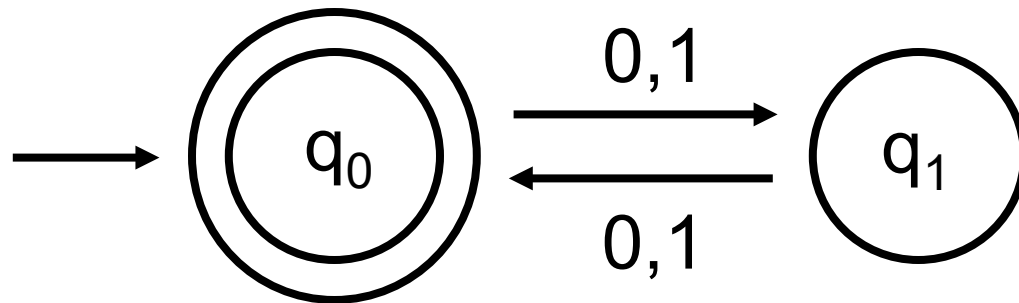
we say  $M$  **decides**  $L$  if  $L(M) = L$ .



What language does this DFA decide?

Strings with an even  
number of 1s

$0^*(10^*10^*)^*$

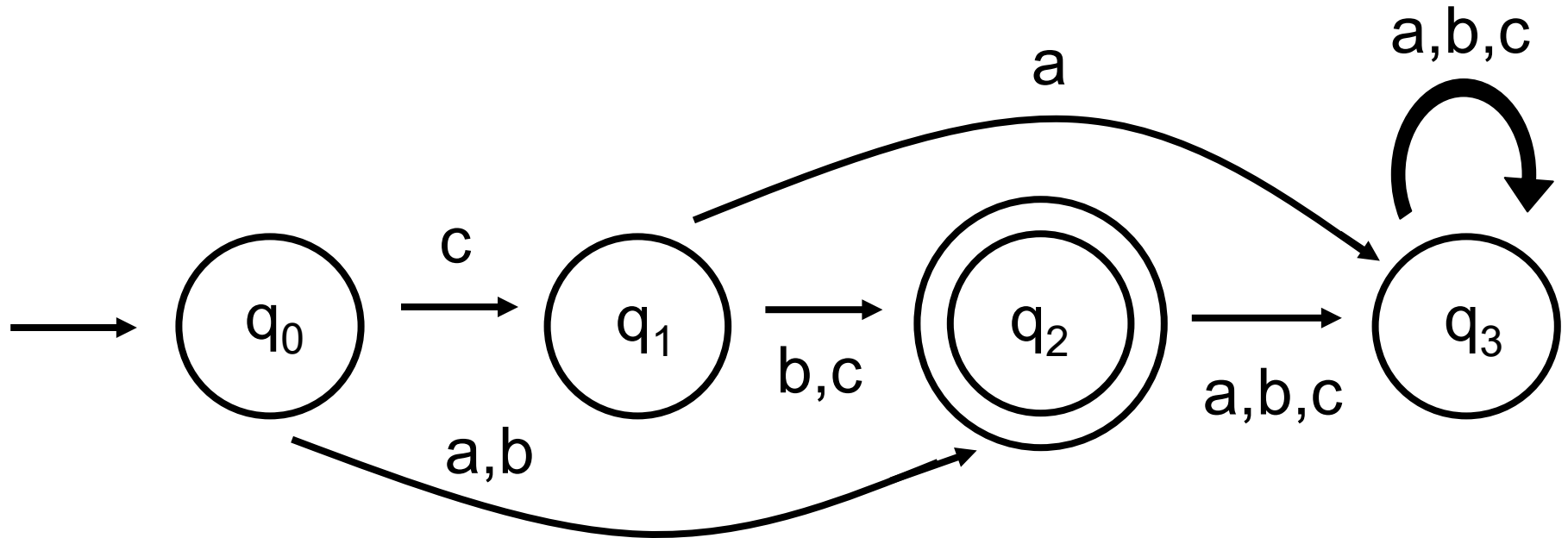


What language does this DFA decide?

Strings with an even  
number of symbols

$((1|0)(1|0))^*$

M is the following DFA  
with alphabet  $\Sigma=\{a,b,c\}$ :

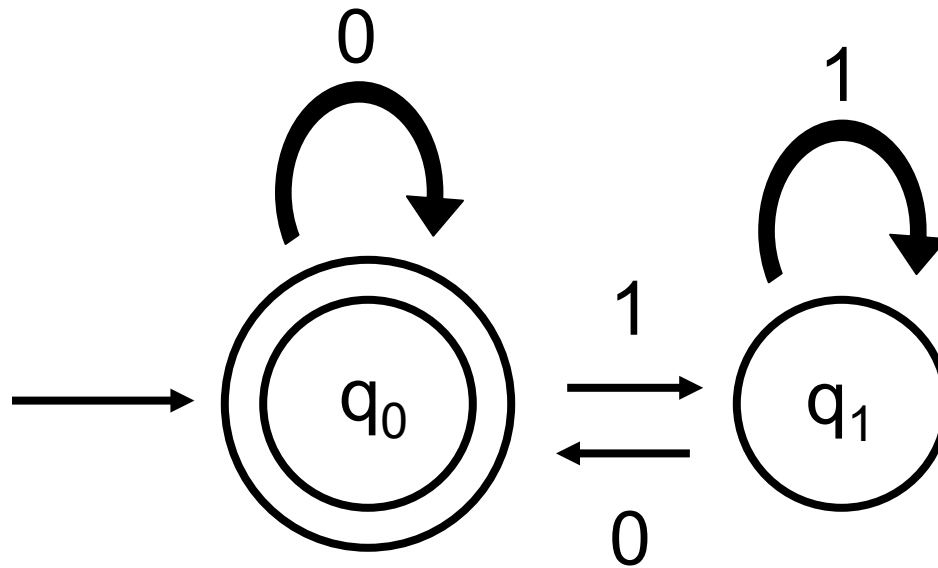


$L(M) =$

$\{a,b,cb,cc\}$

$a \mid b \mid cb \mid cc$

$M$ :



$L(M) =$

Strings that end with 0  
(plus the empty string)

$\epsilon \mid (0|1)^*0$

**Fact:** If there is a DFA that decides a language  $L$ , then there is also a DFA that decides  $L^c$ , the **complement of  $L$** . Why?

E.g.,  
 $L = \{\text{strings containing } 01\}$   
 $L^c = \{\text{strings NOT containing } 01\}$

**Proof:** Given a DFA  $M$  that decides  $L$ ,  
Construct  $M'$  by flipping which states are accepting  
 $M'$  accepts exactly those strings that  $M$  does not, so  $L(M') = L^c$

This strategy is one that will come back many times. We assumed that we had a machine that decides one language, then used that to construct a brand new machine that decides a different language

# Formal definition of DFAs

A **deterministic finite automaton** is a 5-tuple:

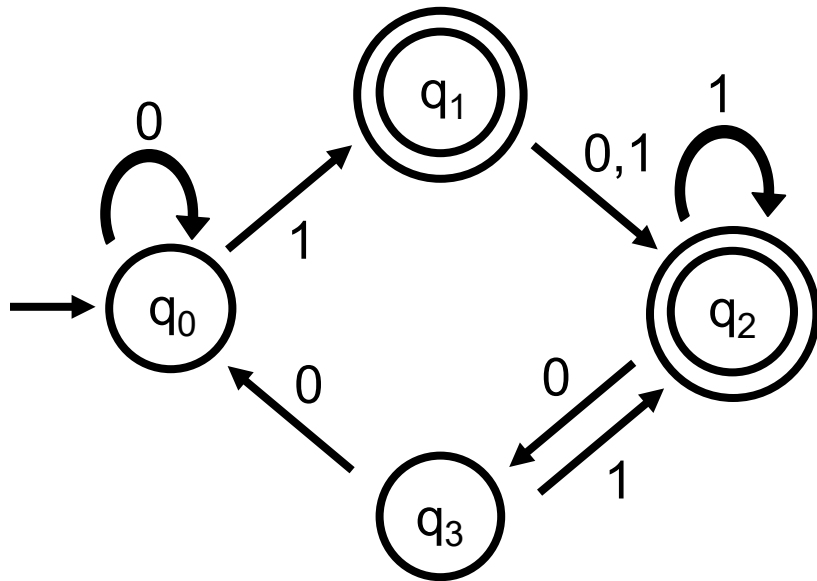
$$M = (\Sigma, Q, q_0, F, \delta)$$

- 1  $\Sigma$  is a (finite) alphabet,
- 2  $Q$  is a nonempty *finite* set of states,
- 3  $q_0 \in Q$  is the start state,
- 4  $F \subseteq Q$  is the set of accepting states,
- 5  $\delta : Q \times \Sigma \rightarrow Q$  is the state-transition function.

# Formal definition of DFAs

A **deterministic finite automaton** is a 5-tuple:

$$M = (\Sigma, Q, q_0, F, \delta)$$



$$\Sigma = \{0,1\}$$

$$Q = \{q_0, q_1, q_2, q_3\}$$

$q_0$  is the start state

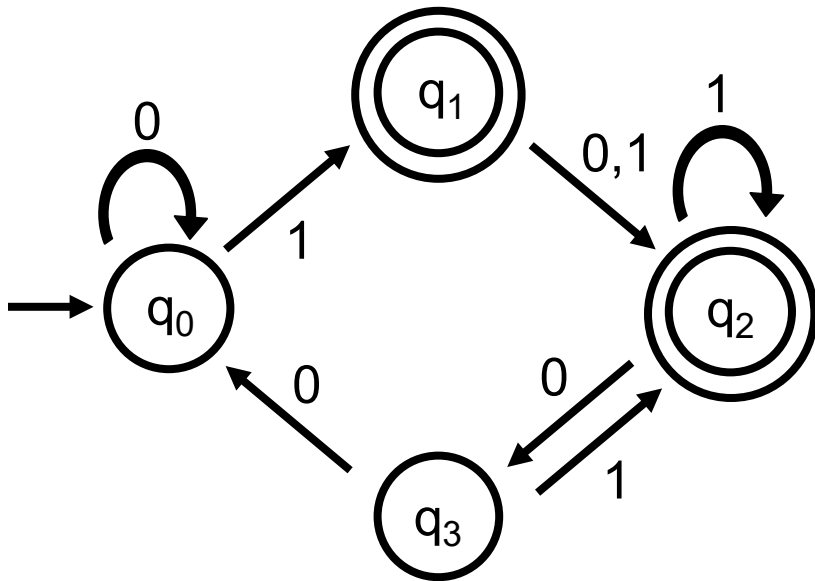
$$F = \{q_1, q_2\}$$

# Formal definition of DFAs

A **deterministic finite automaton** is a 5-tuple:

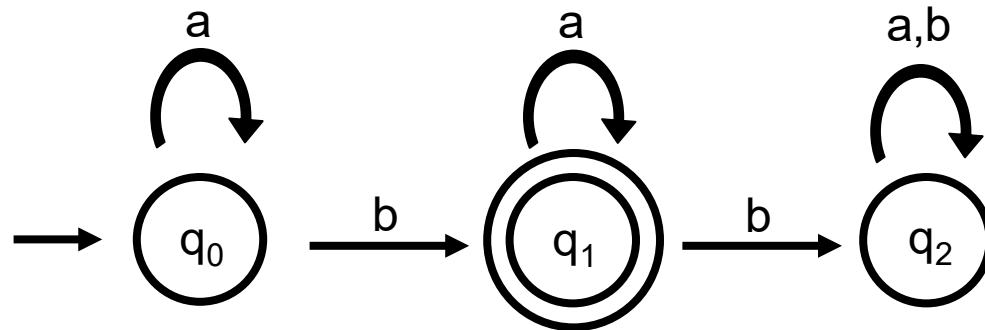
$$M = (\Sigma, Q, q_0, F, \delta)$$

$\delta : Q \times \{0,1\} \rightarrow Q$  is...

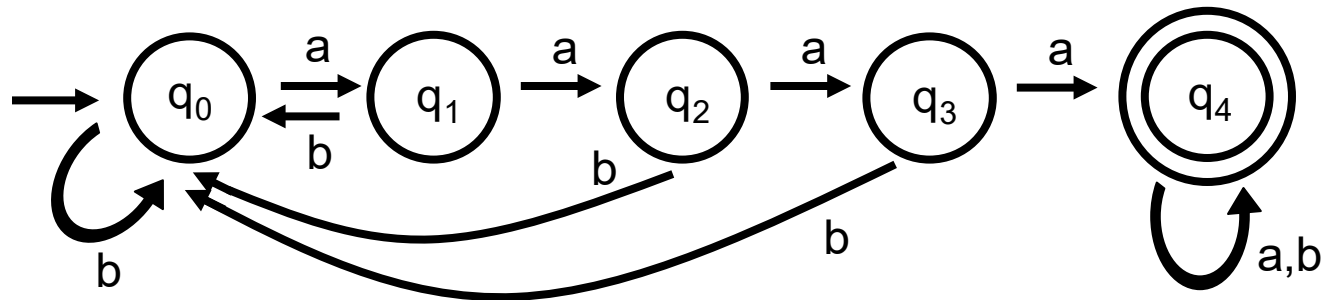


| $\delta$ | 0     | 1     |
|----------|-------|-------|
| $q_0$    | $q_0$ | $q_1$ |
| $q_1$    | $q_2$ | $q_2$ |
| $q_2$    | $q_3$ | $q_2$ |
| $q_3$    | $q_0$ | $q_2$ |

1. For  $\Sigma = \{a,b\}$ , consider the language of strings with exactly one b
  - a. Write using string notation  $a^*ba^*$
  - b. Draw a DFA



2. Draw a DFA for the language of strings that have “aaaa” in them



3. Draw a DFA for  $L = \{x^k y^k : k \geq 0\}$

# Impossibility Proof

Consider *any* DFA **M**. We show **M** does not decide  $L=\{x^k y^k : k \geq 0\}$ , by exhibiting a string on which **M** gives the “wrong” answer.

Any fixed string (e.g. xxxy or xxyy) has a single answer, so we could custom build a DFA that gets that one right. We need our string to depend on **M**

A more powerful DFA can process more, so we would need a bigger instance in order to thwart it

Technique: We will come up with 2 strings with different answers, but prove that M MUST return the same answer for both!

# Impossibility Proof

Consider any DFA **M**. We show **M** does not decide  $L = \{x^k y^k : k \geq 0\}$ ,

Let **s** = #states of **M**. (It's a finite number!)

Consider the (finite) input string  $x^s y^s$ .

As we process the string, the DFA switches to some different states  $q_i$ :

|               |           |           |           |           |           |         |           |
|---------------|-----------|-----------|-----------|-----------|-----------|---------|-----------|
| $\varepsilon$ | $x$       | $xx$      | $xxx$     | $xxxx$    | $xxxxx$   | $\dots$ | $x^s$     |
| $q_{i_0}$     | $q_{i_1}$ | $q_{i_2}$ | $q_{i_3}$ | $q_{i_4}$ | $q_{i_5}$ | $\dots$ | $q_{i_s}$ |



There are **s states in M** (pigeonholes) and **s+1 visits** (pigeons), so there must be *distinct* **i, j**  $\leq s$  such that **state**( $x^i$ ) = **state**( $x^j$ ).

**Claim.** The two inputs  $x^i y^i$  and  $x^j y^i$  have the same final state. Why?

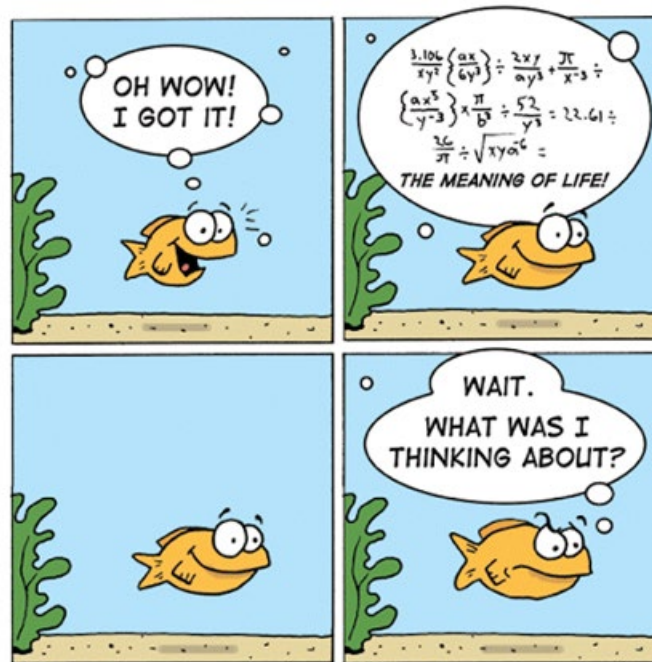
This means **M** must accept both or reject both!

But  $x^i y^i \in L$  and  $x^j y^i \notin L$ , so **M** is wrong on one of these!

**Conclusion:** **M** does not decide L.

Intuitive reason why DFAs cannot decide many languages:

**Fixed (finite) memory!**



SPUDCOMICS.COM

© 2010 LONNIE EASTERLING

THE TRAGEDY OF A THREE SECOND MEMORY

A DFA models a computer with a fixed amount of memory  
(think of the states as “memory”)

**Real computers are only as powerful as DFAs**

If you have only 1TB of storage, there must be a  $k$  (over 2.6 trillion *digits long*) where your computer will not give the right answer

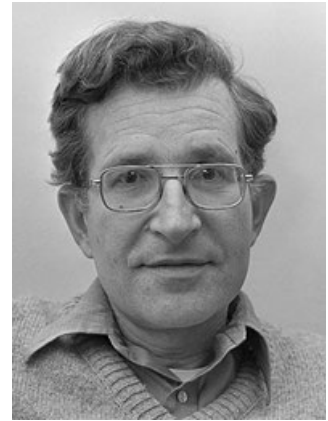
Your laptop also cannot decide the language  $L = \{x^k y^k : k \geq 0\}$ .

But, in our definition of a “computer”, we don’t want to artificially constrain ourselves to a fixed amount of memory.

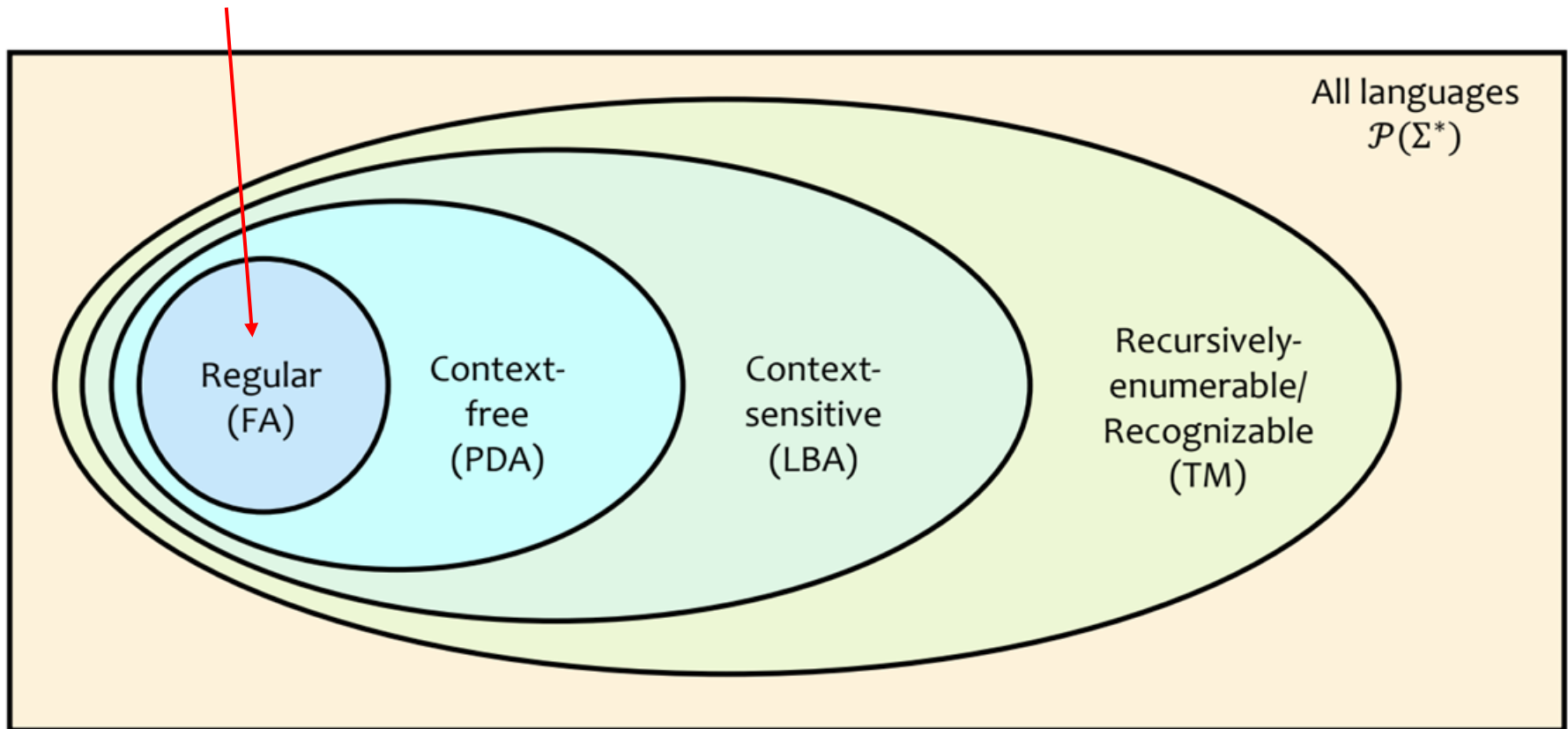
**Instead, we want our definition of a “computer” to model the situation where you can always buy more memory if you need it.**

That’s where **Turing Machines** come in.

# The Chomsky Hierarchy (1956)



**“Regular Language”**: Language decidable by some DFA



More powerful “memory system”  $\longrightarrow$

# DFAs/regularity are widely used in practice

- Simple household devices (switches, garage doors, ...)
- Software for designing and checking the behavior of digital circuits
- “Lexers,” the first stage of compilers
- Flexible string search in text files (logs, latex, ...)
  - o **Warning:** “Regular exprs” in practice often ***aren’t* regular!**
- and much more...